

.mbit File Format Specification (v3.0)

Format Name:	MBIT Compressed Data Stream
Specification Version:	3.0 (DEFLATE Block Architecture)
Core Algorithms:	LZ77 (Sliding Window), Dynamic Huffman (Adaptive Entropy Encoding)
Alignment Type:	Byte-aligned Frame Format

1. Architecture Overview

The **.mbit** format version 3.0 is an industrial-standard lossless binary compression format built on a dual-stage combined **DEFLATE-like** model. Unlike earlier bit-oriented versions, v3.0 utilizes a highly efficient block-parallel structure with a Zero-Allocation policy, minimizing Garbage Collector (GC) pressure and maximizing throughput on modern multi-core processor architectures.

Compression is achieved by partitioning the input stream into independent blocks of a fixed or adaptive size (defaulting to $1\text{ MB} = 1,048,576\text{ bytes}$). Each block is processed through a 4-way heuristic arena to select the most mathematically optimal representation based on marginal encoding density.

2. Global File Structure

An **.mbit** file consists of a fixed 5-byte Global Header followed by a continuous sequence of Data Frames. The file terminates upon reaching the underlying file system's End of File (EOF).

2.1 Global Header

Offset (Byte)	Size (Bytes)	Data Type	Value Description
0 – 3	4	ASCII Characters	Magic Identifier: "MBIT"
4	1	Byte (unsigned)	Format Version Number. For v3.0, the value is <code>0x03</code>

3. Data Frame Anatomy

Each original data block is packed into an isolated frame consisting of a mandatory 9-byte Frame Header and a variable-length Frame Payload. Header fields are aligned in Little-Endian byte order.

3.1 Frame Header

Offset in Frame	Size (Bytes)	Field Name	Technical Description
0	1	ChunkType	Compression algorithm identifier (Arena Mode)
1 – 4	4	CompressedSize	Int32. Exact size of the payload on disk
5 – 8	4	OriginalSize	Int32. Size of the original block after decompression

3.2 Chunk Types (ChunkType)

- **Type 0 (Raw Data):** Direct byte copy. The payload contains raw original bytes. Used for encrypted or high-entropy data where compression yields a negative ratio.
- **Type 1 (LZ77 Only):** Compressed exclusively using a sliding window. Payload consists of literals and 2-byte match pointers.
- **Type 2 (LZ77 + Dynamic Huffman):** Classic DEFLATE approach. Data passes through LZ77, and the resulting stream of flags, literals, and tokens is compressed entirely using a localized Dynamic Huffman tree.
- **Type 3 (Huffman Only):** Pure entropy encoding bypassing LZ77. Applied to files with high character frequency redundancy but low sequential phrasing (e.g., specific logs).

4. Compression Algorithms & Binary Encoding

4.1 LZ77 Subsystem Specification

The LZ77 engine operates with a history sliding window of **4095 ext{bytes}** (12 bits). The minimum match length to be encoded is $L_{\{min\}} = 3 \text{ ext{bytes}}$, and the maximum is $L_{\{max\}} = 18 \text{ ext{bytes}}$ (4 bits).

The stream is encoded in groups of 8 tokens. Each group is preceded by a single **Flag Byte**, where each bit (from LSB to MSB) corresponds to the type of the next token:

- Bit equals **0**: The token is a **Literal** (1 raw byte).
- Bit equals **1**: The token is a **Match Pointer** (2 bytes / 16 bits).

16-bit Pointer Structure: `[Distance: 12 bits] | [Length: 4 bits]`. The length is stored with an offset: `ext{Code} = L - L_{min}`, allowing the 4 bits to encode the range 3 to 18.

Hardware Acceleration Scanning

For $O(1)$ match lookups, **Fibonacci Hashing (Golden Ratio Hash)** of 32-bit integers is used with the multiplier `0x9E3779B1`. The engine employs a 64-bit vectorized memory scanner that compares 8 bytes per CPU cycle via `ext{ulong}` casting.

4.2 Dynamic Huffman Specification

Each frame of Type 2 and 3 begins with a unique local prefix dictionary. The dictionary is compacted: a `ext{UInt16}` value indicating the number of distinct bytes in the tree (`ext{Distinct}`). Following this, a triad is written for each symbol: `[Symbol (1 Byte)] + [Code Length (1 Byte)] + [Huffman Code (4 Bytes, UInt32)]`.

The payload bits are packed tightly into a byte array from left to right (Most Significant Bit first).

Mathematical Proof of Safe Bit-Packing

In the theoretical worst-case scenario (Fibonacci frequencies), a Huffman tree degenerates into a linear linked list. To reach a code depth D , the block size N must satisfy the inequality $N \geq F(D)$. Since our block size is strictly limited to `1,048,576 ext{bytes}`, and the 31st Fibonacci number $F(31) = 1,346,269$, it is mathematically proven that **no Huffman code length in this format can exceed 31 bits**. This guarantees zero risk of 32-bit register overflow during bit-packing.

5. Edge Cases & Fault Tolerance

Format v3.0 features hardware-level short-circuiting for edge states:

- **0-Byte Files:** The compressor immediately halts threading and writes exactly 5 bytes to disk: `0x4D 0x42 0x49 0x54 0x03`. The decompressor instantly yields an empty file and terminates cleanly.
- **Files < 4 Bytes:** Minimum window hashing requires 4 bytes. Such micro-blocks are automatically marked as `ChunkType = 0` to prevent out-of-bounds array access.

6. Production & Concurrency Optimizations

1. **Ordered Multi-threading:** Block reads from disk are sequential, but compression for each discrete block occurs in parallel on isolated CPU cores via the `ext{Task}` API. RAM footprint is restricted by a `SemaphoreSlim`. A "Sliding Task Window" pattern queue guarantees the correct deterministic sequence of output frames.

2. **Zero-Allocation:** Hot-path loops are completely devoid of `new byte[]` allocations. Block buffers are rented from a global `ArrayPool<byte>.Shared`. Rental safety is enforced via terminal `try...finally` blocks, neutralizing memory leak risks during critical I/O exceptions.